Seer: Online Context Learning for Fast Synchronous LLM Reinforcement Learning

Ruoyu Qin[†◇] Weiran He[†] Weixiao Huang[†] Yangkun Zhang[†] Yikai Zhao[†]
Bo Pang[†] Xinran Xu[†] Yingdi Shan[◇] Yongwei Wu[◇] Mingxing Zhang[◇1]
[†]Moonshot AI [◇]Tsinghua University

arXiv:2511.14617v1 [cs.DC] 18 Nov 2025

Abstract

Reinforcement Learning (RL) has become critical for advancing modern Large Language Models (LLMs), yet existing synchronous RL systems face severe performance bottlenecks. The rollout phase, which dominates end-to-end iteration time, suffers from substantial long-tail latency and poor resource utilization due to inherent workload imbalance. We present Seer, a novel online context learning system that addresses these challenges by exploiting previously overlooked similarities in output lengths and generation patterns among requests sharing the same prompt. Seer introduces three key techniques: divided rollout for dynamic load balancing, context-aware scheduling, and adaptive grouped speculative decoding. Together, these mechanisms substantially reduce long-tail latency and improve resource efficiency during rollout. Evaluations on production-grade RL workloads demonstrate that Seer improves end-to-end rollout throughput by 74% to 97% and reduces long-tail latency by 75% to 93% compared to state-of-the-art synchronous RL systems, significantly accelerating RL training iterations.

1 Introduction

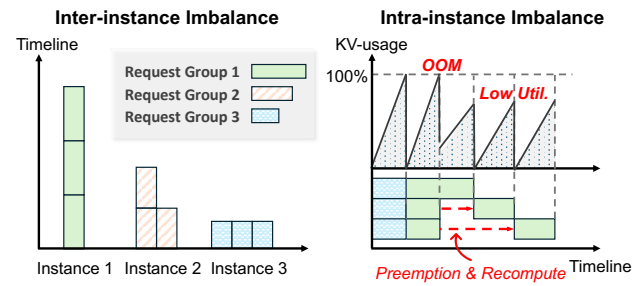
Reinforcement Learning (RL) has become a cornerstone in the development of state-of-the-art Large Language Models (LLMs), enabling significant breakthroughs in complex reasoning and problem-solving capabilities [10,36,37]. The iterative RL training process alternates between a *rollout phase* for data generation and a *training phase* for updating model parameters. However, the rollout phase consistently emerges as the dominant bottleneck, consuming approximately 80% of the total iteration time (see Table 1). Therefore, improving the efficiency of the rollout phase represents one of the most pressing challenges in modern LLM development.

The primary challenge in RL rollout stems from severe resource inefficiency caused by the growing demand for long-generation capabilities, particularly in applications such as complex chain-of-thought (CoT) reasoning [10,37,44]. This creates two critical bottlenecks. First, long-generation tasks, where responses can reach tens of thousands of tokens, exhibit highly unpredictable and rapidly growing memory footprints. A long-CoT request may begin with a footprint of only a few hundred megabytes but expand to tens of gigabytes as decoding progresses. Within a single rollout instance, multiple requests execute concurrently, each with distinct and dynamically evolving memory demands that are difficult to predict. This volatility forces dynamic batch-size shrinkage during rollout, leading to suboptimal hardware utilization or costly preemptions that trigger expensive re-prefills, ultimately degrading overall rollout throughput. Second, these long-generation requests inherently create a heavy-tailed length distribution, leading to severe load imbalance both across and within instances. As a result, toward the end of a rollout iteration, only a handful of long-running requests remain active on a few instances, drastically reducing accelerator occupancy. This long-tail inefficiency is so pronounced that utilizing resources from idle nodes for long-tail requests often fails to provide significant speedups and can even degrade performance due to increased inter-node communica-

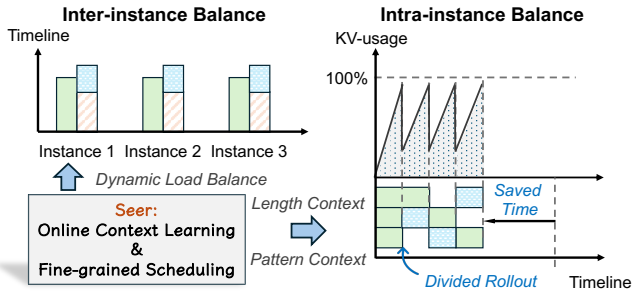
Table 1: Time distribution across RL training phases for different workloads. All models are trained as reasoning models with chain-of-thought capabilities, where Moonlight [25] and Kimi-K2 [36] are trained on mathematical datasets, while Qwen2-VL-72B [42] is trained on language-vision-mixed reasoning tasks using the LLM-as-a-Judge [34] reward model. Reward computation is performed by a dedicated reward server and overlaps nearly completely with the rollout phase. Detailed workload configurations are described in §6.1.

	Rollout	Training	Weight Update
Moonlight	84%	14%	2%
Qwen2-VL-72B	63%	31%	6%
Kimi-K2	87%	10%	3%

[1] Corresponding to zhang_mingxing@mail.tsinghua.edu.cn.



(a) Group-level rollout.



(b) Seer's rollout.

Figure 1: Challenges and Seer's solution for long-generation rollout. Conventional group-level rollout treats request groups as monolithic units, leading to severe inter-instance and intra-instance load imbalance. Seer achieves dynamic load balancing and prevents preemption through divided rollout. Building upon divided rollout, Seer implements online context learning, enabling context-aware scheduling and adaptive grouped speculative decoding to further reduce rollout time.

tion overhead.

To improve hardware utilization, recent works have explored asynchronous rollout systems [6, 11, 31, 51], which overlap the rollout and training phases. While this approach can reduce end-to-end iteration time, it comes at the cost of algorithmic fidelity. By nature, these systems introduce a degree of off-policy learning, as data generated with model parameters from step i may be used to train the model at step $i+1$ or beyond. Furthermore, asynchronous or partially asynchronous [9, 13, 53] RL systems often suffer from distributional skew, where faster-to-generate short samples disproportionately populate early training batches [38]. These off-policy effects can diminish final model performance and complicate debugging and reproducibility [12]. Consequently, synchronous (or "on-policy") rollout remains critical in many settings for ensuring methodical evaluation, reproducibility, and strict adherence to the underlying algorithm's assumptions. This paper, therefore, focuses on optimizing the synchronous case, although we note that our techniques can also be adapted to improve asynchronous settings.

Speculative decoding [17] (SD) presents another promising direction for accelerating memory-bound generation. However, conventional SD, which relies on a small, static draft model, is incompatible with the RL training loop. Existing model-based speculation methods face two critical limitations in the RL context: (1) draft models obtained through distillation or incremental fine-tuning struggle to adapt to continuously evolving target models during iterative RL updates, and (2) draft generation incurs significant computational overhead (millisecond-level latency) and GPU memory footprint, which become prohibitive at large batch sizes.

To address these challenges, we introduce **Seer**, a novel system for synchronous RL rollout that dynamically exploits contextual information within the workload to maximize resource efficiency. Seer is built upon a key observation: popular RL algorithms such as Group Relative Policy Optimization [29] (GRPO) generate G (typically 8–16) responses per prompt, and responses within a group tend to exhibit similar length profiles and recurring local token patterns, which represent a rich structure that existing schedulers and inference engines leave untapped. For instance, generating an early "probe" response per prompt provides a strong, online estimator of that prompt group's remaining work (i.e., expected output length and KVCache footprint), enabling more informed scheduling decisions than static heuristics. Seer leverages this latent intra-group context to make three key contributions:

1) Divided Rollout with Global KVCache: Seer breaks away from the conventional group-level scheduling paradigm in which entire request groups (i.e., requests sharing the same prompt in GRPO-like algorithms) are dispatched as monolithic units. Such group-level scheduling leads to severe inter-instance and intra-instance load imbalance, as illustrated in Figure 1. Instead, Seer implements *Divided Rollout* to achieve dynamic load balancing and maximize VRAM utilization. A request group is not only split into G independent requests but further decomposed into multiple chunks that are scheduled and dispatched incrementally. This strategy enables the entire rollout process to maximize resource utilization without triggering costly preemptions. For example, the scheduler can pack a large number of shorter requests into initial batches to maximize VRAM utilization early on, while later, when long-running requests dominate, it dynamically controls concurrency by computing KVCache budgets. To enable dynamic load balancing, Seer's global scheduler monitors instance-level KVCache usage and can migrate a request to a less-loaded worker upon its next scheduling. This migration is made efficient by a global KVCache pool, inherited from Mooncake [28], shared across instances and backed by DRAM/SSD, which eliminates the expensive overhead of prefill recomputation.

2) Context-Aware Scheduling: Seer leverages a "speculative request" mechanism to inform its scheduling policy. By generating one response from each GRPO group at high priority, it obtains an online estimate of the group's expected generation length and KVCache footprint. This allows the global scheduler to implement an approximate longest-job-first pol-

icy, prioritizing longer tasks to run alongside shorter ones and thus maximizing batch density. Experiments in §6.4.1 show that this approach significantly reduces the time spent in the long-tail phase.

3) Adaptive Grouped Speculative Decoding: To leverage speculative decoding for rollout acceleration while overcoming the acceptance rate collapse of traditional SD, Seer introduces an online context learning-based speculative mechanism. Seer deploys a Distributed Grouped Draft Server (DGDS) that maintains a Compressed Suffix Tree [45] (CST) for each group, aggregating token sequences from all requests within the same group. This approach creates a highly accurate, dynamic "draft model" that is inherently synchronized with the target model. Additionally, DGDS introduces an adaptive draft range mechanism to maximize system throughput. Our ablation experiments in §6.4.2 demonstrate the effectiveness of both key components. Using the system without speculative decoding as the baseline, our adaptive grouped speculative decoding outperforms two ablation variants by 27% and 11% in end-to-end throughput, which correspond to disabling the adaptive strategy and disabling group context information, respectively.

We have implemented Seer and evaluated it on production-grade RL workloads. Experimental results show that Seer achieves throughput improvements ranging from 74% to 97% and reduces long-tail latency by 75% to 93% compared to a highly optimized baseline, establishing a new state-of-the-art for efficient, synchronous RL infrastructure.

2 Background and Motivation

2.1 RL for Reasoning LLMs

Reinforcement learning (RL) training for large language models (LLMs) is an iterative process where the model learns from self-generated responses to improve its policy. Each RL iteration consists of the following logical phases: (1) `Rollout`, where the model generates response trajectories based on a given prompt set. In mainstream algorithms like Group Relative Policy Optimization [29] (GRPO), each prompt produces 8–16 responses; (2) `Reward computation`, where a reward server (rule-based [10], sandbox [4], or LLM-as-a-Judge [34]) evaluates the generated responses; (3) `Experience collection`, where algorithms such as GRPO compute the training experiences needed for policy updates (detailed in §3.1); (4) `Training`, where the model are trained using the collected experiences; and (5) `Weight update`, where the updated weights are propagated to the inference model in preparation for the next rollout iteration.

Many recent works [26,36,52] have optimized this pipeline while maintaining the logical ordering of each iteration (known as *synchronous RL training*). For example, RLHFuse [52] overlaps the long-tail phase of rollout with reward computation and experience collection to reduce pipeline bubbles, while Kimi-K2 [36] implements efficient model offloading and checkpoint updates. However, the *rollout phase*, the dominant bottleneck in RL training, has not been adequately addressed. As shown in Table 1, the rollout phase accounts for 63–87% of the total RL training time. Due to the extremely long and highly variable output lengths of reasoning LLMs, the rollout phase suffers from severe resource underutilization, making it the primary target for optimization.

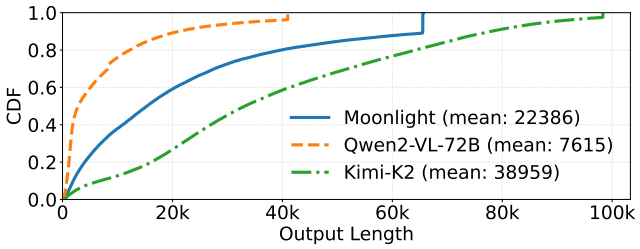


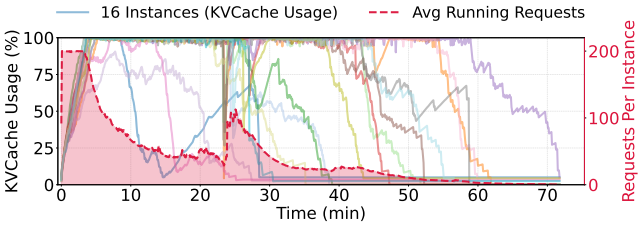
Figure 2: Distribution of output lengths during rollout across three reasoning tasks. The generation lengths span from hundreds to 98K tokens, demonstrating both high average length and extreme variance.

2.2 Key Challenges in Rollout

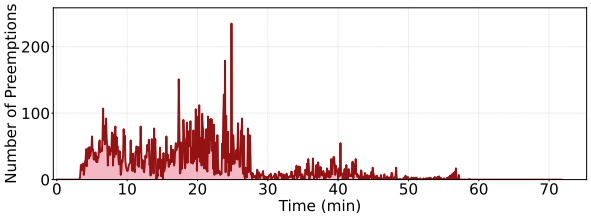
Training chain-of-thought (CoT) reasoning is central to RL for reasoning models [10, 37]. Models are trained to generate longer and more detailed reasoning chains to enhance their problem-solving capabilities. However, this introduces two critical characteristics during rollout: (1) **long average output lengths**, and (2) **high variance in output lengths across requests**. Figure 2 illustrates the distribution of output lengths during rollout for three tasks, revealing that generation lengths span an enormous range from a few hundred tokens to as many as 98K tokens. On one hand, the long average output length (tens of thousands of tokens) places immense pressure on memory management. On the other hand, the high variance in output lengths leads to long-tail inefficiency, where a small number of extremely long requests monopolize GPU resources in the later stage of rollout.

Figure 3 further demonstrates the resource waste caused by long generation sequences during rollout. These metrics reveal two major challenges:

Challenge #1: Varying memory consumption. For requests with average output lengths of tens of thousands of tokens, the KVCache can occupy several gigabytes of memory. This memory footprint is amplified by a factor of G in GRPO-like algorithms, where G responses are generated for each prompt within the same group. Such massive memory consumption can lead to memory exhaustion, forcing the system to preempt running requests. In the worst case, a single instance can only execute one prompt group at a time. However, reserving KVCache space upfront to avoid preemption leads to severe resource underutilization. Since memory consumption



(a) KVCache utilization and average number of running requests across instances.



(b) Request preemption count.

Figure 3: KVCache utilization, number of running requests, and preemption count during a synchronous rollout phase of the Qwen2-VL-72B task. In the early stage of rollout, insufficient KVCache capacity causes frequent request preemptions; in the later stage, a small number of extremely long request groups contribute to a long-tail period that accounts for nearly half of the total rollout time.

is proportional to the total sequence length, requests in their early generation phase (with only a few hundred tokens) may occupy the entire memory space for hundreds of seconds, drastically reducing throughput.

Challenge #2: Severe long-tail effect. The long-tail problem is another critical issue in rollout, widely noted in prior work [9, 13, 53]. As shown in Figure 3, the long-tail phase can account for nearly 50% of the total rollout time. This phenomenon arises from two factors: (1) In GRPO-like algorithms, requests within the same group tend to have similar lengths (see §3.2). Groups with extremely long average lengths form "monolithic" batches that cause severe load imbalance across instances when scheduling is performed at the group granularity. (2) Under memory constraints, requests may be preempted or delayed, causing extremely long requests to be blocked and further exacerbating scheduling delays.

To mitigate the long-tail effect and improve rollout scalability, recent works [6, 9, 11, 13, 31, 51, 53] have proposed *asynchronous RL*. Unlike synchronous methods, where training experiences must all originate from the current policy, asynchronous methods allow training on stale rollout data from previous policy iterations, introducing *off-policyness*. By staging long-tail rollouts in advance or repacking requests, asynchronous RL can significantly reduce long-tail impact. However, to achieve the best convergence and maintain reward stability, many RL experiments still prefer synchronous training.

Speculative decoding (SD) offers a promising approach to accelerate the long-tail phase while preserving synchronous RL training guarantees. Existing SD techniques generally fall into two categories: model-based and model-free methods. Model-based SD approaches face key limitations in RL scenarios due to their static draft models and substantial overhead. First, the target LLM in RL is continuously updated, causing rapid "model drift". This divergence quickly renders the static draft model's predictions obsolete, leading to a declining acceptance rate that nullifies potential performance gains. Second, the draft model itself introduces non-negligible latency and GPU memory consumption. In rollout scenarios characterized by large-batch offline inference workloads, such overhead becomes prohibitive. For model-free SD methods, n-gram-based approaches [7, 39] are commonly employed. However, n-gram techniques require matching against the entire input sequence when generating draft tokens, incurring substantial overhead in long-context scenarios. Furthermore, n-gram methods struggle to aggregate multiple potential matches with low computational complexity, resulting in suboptimal draft token generation. To address these limitations of existing SD approaches, we propose grouped speculative decoding based on compressed suffix trees [45]. As a model-free method, it significantly reduces draft complexity while leveraging pattern similarity among requests within the same group to generate high-quality draft tokens. The detailed design of our SD approach is presented in §4.4.

Summary. In synchronous LLM RL training, the rollout phase faces severe challenges in memory management and long-tail resource waste due to the extremely long CoT reasoning process. This is because existing RL systems treat each prompt group as a monolithic unit, ignoring the length and pattern similarities within groups. Our system, Seer, takes a different approach by decomposing prompt groups into individual responses and further down to finer-grained chunks for scheduling. By leveraging the similarity among responses within the same group, Seer optimizes both scheduling and speculative decoding, achieving lossless acceleration of the rollout phase.

3 Characterizing Contextual Signals in Rollout

3.1 Group Property of GRPO-like Algorithms

In LLM reinforcement learning practice, the most widely adopted algorithm is Group Relative Policy Optimization [10] (GRPO). GRPO performs group-based preference optimization without the need for a value network (critic). Its core idea is to sample G candidate responses $\{y_i\}_{i=1}^{G}$ for each prompt from the current policy $\pi_{\theta_{old}}$, evaluate them using a rule-based or learned reward model to obtain rewards $\{r_i\}$, normalize

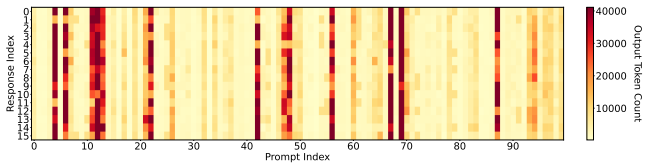


Figure 4: Length correlation within response groups. Each column represents a prompt group in GRPO rollout, and each cell corresponds to an individual response. The color intensity indicates output length. Responses within the same group exhibit strong length correlation, with most groups showing similar generation lengths across their requests.

these rewards within the group to compute the advantages $\{A_i\}$, and finally update the policy π_θ with a clipped ratio loss and KL regularization toward a reference model π_{ref} to stabilize training. The objective can be expressed as:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_{x,y_i\sim\pi_{\theta_{\text{old}}}}\left[\min\Big(\rho_i A_i,\ \text{clip}(\rho_i,\,1-\varepsilon,\,1+\varepsilon)A_i\Big)\right] + \beta\,\text{KL}(\pi_\theta \,\|\, \pi_{\text{ref}}),\ \rho_i = \frac{\pi_\theta(y_i\,|\,x)}{\pi_{\theta_{\text{old}}}(y_i\,|\,x)}$$

where ε and β are hyperparameters, and A_i is the advantage, computed using the *group* of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the G outputs within each group:

$$A_i = \frac{r_i - \text{mean}\big(\{r_1, r_2, \ldots, r_G\}\big)}{\text{std}\big(\{r_1, r_2, \ldots, r_G\}\big)}.$$

Several recent works [48, 49] have refined GRPO to improve generalization, but they all preserve the core principle of generating multiple responses for the same prompt. Some recent efforts, such as BroRL [14] and Knapsack RL [21], have even proposed scaling G to 512 or more to enhance exploration capabilities. This training paradigm naturally introduces a strong and stable contextual signal during rollout, as responses in the same group exhibit notable similarity in semantics, structural templates, and generation length.

However, existing RL systems typically treat each prompt group as a monolithic unit during rollout, overlooking the potential to externalize and exploit such intra-group similarity as **shared contextual information**. By explicitly leveraging this shared context during synchronous RL rollout, it becomes possible to (i) achieve length- and memory-aware scheduling, and (ii) enable grouped speculative decoding that exploits sequences from co-grouped requests to accelerate inference, thereby mitigating long-tail inefficiency and improving overall throughput without violating the on-policy training constraint.

3.2 Length Context for Efficient Scheduling

As a set of responses generated from the same input, the generation lengths of responses within a group are highly predictable. In general, the length of model responses reflects the difficulty of the prompt. Prior work [5,8,51] has attempted to predict generation length using small models or fitting methods based on prompt characteristics, which demonstrates that for the same model and prompt, response lengths exhibit strong correlation. Figure 4 illustrates this length correlation across multiple prompt groups in RL rollout, where most groups show consistent lengths among their responses.

This property enables us to obtain length information through speculative sampling, which can then inform global scheduling decisions. Specifically, to address the long-tail problem, longest-job-first scheduling (LFS) is a commonly employed strategy. In the context of rollout, we can prioritize scheduling longer requests to minimize long-tail latency. Such a scheduling policy requires knowledge of request generation lengths. The length context inherent in grouped rollout provides a higher-quality source for length prediction, forming the foundation for the approximate LFS scheduling algorithm (detailed in §4.3).

Table 2: Mean acceptance length in n-gram speculative decoding with grouped pattern references under different draft strategies. We sample 20 prompt groups from the Qwen2-VL-72B task and simulate speculative decoding using CSTs (maximum draft length of 8 tokens per step). The reference count n indicates how many other responses from the same group are included in the CST. Linear generates one draft sequence per step, while multi-path generates multiple candidate sequences with top-k branching. Values represent mean acceptance length (including bonus token).

Ref. Count	**Linear**	**Multi-Path (k=2)**	**Multi-Path (k=4)**
$n = 0$ (baseline)	1.70	1.77	1.85
$n = 1$	2.04	2.14	2.25
$n = 5$	2.32	2.44	2.59
$n = 15$	2.53	2.69	2.85

3.3 Pattern Context for Efficient Inference

In addition to exploiting length correlation for scheduling, the grouped nature of GRPO rollout enables another optimization opportunity: pattern-level similarity. Since all responses within a group are conditioned on the same prompt, they naturally exhibit semantic and structural similarities in the generated content. While this similarity is not as precise as exact prefix matching [28, 50], it provides sufficient signal to construct a grouped pattern dictionary for more effective n-gram-based speculative decoding.

To quantify this intuition, we conducted an empirical study on real RL workloads. We sampled 20 prompt groups and simulated n-gram speculative decoding using compressed suffix trees (CSTs). Unlike conventional lookahead decoding, which relies solely on each request's own generation history

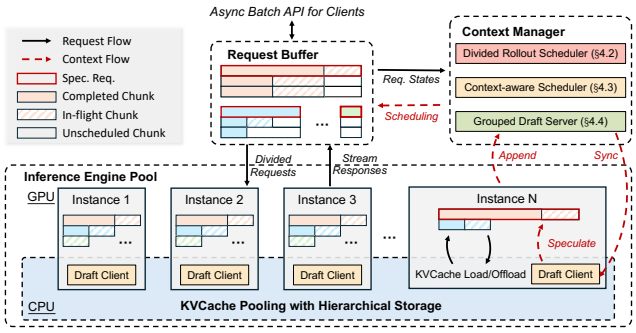


Figure 5: The overview of Seer.

as the n-gram dictionary, our approach incorporates other responses from the same group as reference patterns to measure cross-request similarity. Table 2 presents the results. Compared to the baseline ($n = 0$), introducing grouped pattern references improves the number of accepted draft tokens (excluding the bonus token) by up to 119%, while multi-path drafting further enhances efficiency by generating multiple candidate sequences per step. Leveraging this insight, Seer incorporates a distributed grouped draft server that aggregates pattern context across all requests in a group using an efficient CST structure, enabling both high-quality and efficient speculative decoding. In practice, Seer dynamically adjusts key parameters, including the maximum draft length and the number of candidate paths (k), based on workload characteristics and resource availability to maximize rollout throughput.

4 Design of Seer

4.1 Overview

Seer is an efficient synchronous RL framework specifically designed to accelerate the rollout phase. As motivated by the workload analysis in §3, Seer enables online context learning during rollout to achieve efficient scheduling and inference. In addition to context-aware optimizations, Seer incorporates efficient memory management, load balancing, and asynchronous reward computation. Despite these performance enhancements, Seer maintains logical consistency across all stages of the synchronous RL pipeline, thereby achieving an algorithmically lossless reinforcement learning process.

Figure 5 illustrates the architecture of Seer's rollout system, which comprises three core modules: the `Inference Engine Pool`, the `Request Buffer`, and the `Context Manager`. The Inference Engine Pool consists of inference instances and a distributed KVCache pool across inference nodes, which are responsible for executing inference requests with load balancing and cache reuse capabilities. The Request Buffer serves as the unified entry point for all requests, managing their inputs, outputs, and runtime states. The Context Manager maintains contextual views for all requests and provides scheduling decisions based on request contexts.

To address the challenges identified in §2.2, Seer introduces three key techniques. First, to tackle the load imbalance problem (Challenge #1), Seer proposes **Divided Rollout** (§4.2), which leverages the Request Buffer and the global KVCache pool to enable fine-grained load balancing at the sub-request level. Second, to mitigate the long-tail effect (Challenge #2), Seer implements **Context-Aware Scheduling** (§4.3), which utilizes length context within request groups to reduce long-tail latency. Third, Seer employs **Adaptive Grouped Speculative Decoding** (§4.4), which exploits pattern context within request groups to further accelerate inference, particularly for long requests.

4.2 Divided Rollout

In conventional rollout systems, inference instances process requests at the group level, either in an offline batch mode or an online server mode. This approach typically distributes a large batch of request groups (where each group consists of G requests sharing the same prompt) randomly to inference instances. Once a request is assigned to an inference instance, it remains bound to that instance until completion. However, this design suffers from two critical limitations. First, since different request groups exhibit vastly different generation lengths, leading to severe load imbalance across instances and memory management challenges within individual instances, as discussed in §2.2. Second, this method treats each group as a monolithic unit, disregarding the contextual similarities between requests within the same group, which could otherwise be exploited for optimization as discussed in §4.3 and §4.4.

To address these limitations, Seer builds upon the Request Buffer to implement a fine-grained request and concurrency control mechanism called **Divided Rollout**. For a monolithic request group where all requests share the same prompt, Seer not only decomposes it into G individual requests but further divides each request into multiple chunks based on generation length. This sub-request-level rollout is managed by the Request Buffer, which maintains comprehensive metadata for each request, including its group ID, prompt length, original `max_tokens`, and current generated length. When a request is dispatched from the Request Buffer according to the scheduling policy (described in §4.3), its `max_tokens` is set to a small chunk size (e.g., 8K tokens). After completion of the current chunk, the request is re-enqueued into the Request Buffer and resubmitted iteratively until it generates an `<eos>` token or reaches its original `max_tokens`.

With divided rollout, Seer achieves more efficient scheduling both within and across inference instances, as illustrated in Figure 1b:

Fine-grained Memory Management. Conventional scheduling approaches face a trade-off in rollout scenarios: high

concurrency leads to preemption in the later stage, while low concurrency results in resource underutilization in the early stage. This dilemma becomes more severe as the average output length increases. Through divided rollout, each request generates tokens in significantly smaller chunks, thereby maintaining a nearly constant KVCache footprint throughout the generation process. Seer's scheduler can dynamically compute the maximum concurrency level that avoids triggering preemption to maximize resource utilization.

Dynamic Load Balancing. Divided rollout transforms the scheduling granularity from a single instance selection per request group to $G \times num_chunks$ selections, enabling dynamic load balancing across instances. When a sub-request is resubmitted from the Request Buffer, Seer dynamically selects the least-loaded inference instance based on real-time monitoring of in-flight request concurrency and their corresponding memory footprints. This significantly reduces the long-tail effect caused by load imbalance across instances.

To support efficient divided rollout while avoiding redundant KVCache recomputation during re-dispatch, Seer builds upon Mooncake [28] to construct a globally shared KVCache pool distributed across inference nodes. Seer's divided rollout acts as a proactive request migration and load-balancing mechanism that is transparent to the upper layer, specifically designed for offline inference workloads. Compared with passive KVCache offloading triggered by preemption, this approach does not block the inference process. Compared with live migration systems for online serving (e.g., Llumnix [35]), it offers greater scheduling flexibility due to the absence of strict per-request SLO constraints. Based on divided rollout, Seer enables more flexible request scheduling that particularly benefits from exploiting group contextual information, as detailed in §4.3.

4.3 Context-Aware Scheduling

Beyond inter-instance load imbalance, intra-instance scheduling imbalance also contributes to long-tail latency during rollout. Within a single instance, memory constraints force some requests to be delayed. Under naive scheduling policies, long requests may be deferred until the end, further exacerbating the long-tail effect. As analyzed in §3.2, requests within the same prompt group tend to exhibit highly correlated output lengths. This observation motivates Seer's scheduling design: leveraging length context across grouped requests to predict output lengths and implement approximate longest-first scheduling (LFS).

Based on divided rollout, Seer designates the first request of each group as a `speculative request`, which acts as an online probe to estimate the expected workload of the remaining requests in that group. Speculative requests are assigned higher scheduling priority and follow a shortest-first scheduling (SFS) approach that prioritizes requests with shorter generated lengths. This **length filtering** approach exploits the fact that short requests complete quickly, allowing Seer to rapidly identify long-tail samples early in the rollout and dynamically update length information to the Context Manager in real time. Based on these updated length estimates, Seer schedules the remaining requests using an approximate LFS policy that prioritizes groups with longer predicted generation lengths.

Algorithm 1 Context-Aware Scheduling based on Divided Rollout

Require: Active requests $\mathcal{R} = \{r_{g,i}\}$ grouped by prompt g; group-level length estimates $\{\widehat{L}_g\}$; inference instances I with KV-usage telemetry.
Ensure: A scheduling decision $(r^\star, i^\star)$ with $r^\star \in \mathcal{R}$ and $i^\star \in I$.

```
for all r_{g,i} ∈ R do
    if r_{g,i} is finished then
        L̂_g ← UpdateEstimate(L̂_g, L_{g,i})
        remove r_{g,i} from R
    else if r_{g,i} is the group's speculative request then
        keep in high-priority queue Q_spec
    else
        add to low-priority candidate set C_rest
    end if
end for
r* ← None
if ¬IsEmpty(Q_spec) then
    r* ← PickSFS(Q_spec)        ▷ SFS: smallest generated length first
else if ¬IsEmpty(C_rest) then
    r* ← PickLFS(C_rest)        ▷ LFS: largest L̂_g first
else
    return all requests are finished
end if
r*.max_tokens ← min(chunk_size,
                    r*.ori_max_tokens − r*.generated_tokens)
i* ← SelectInstance(I, r*.max_chunk_tokens, KV-usage)
if i* ≠ None then
    return (r*, i*)
end if
return no available instance for this cycle
```

Algorithm 1 presents the context-aware scheduling workflow built on top of divided rollout. The algorithm is invoked continuously by Seer's global scheduler, each time returning a scheduling decision $(r^\star, i^\star)$ that assigns a selected request $r^\star$ to an inference instance $i^\star$, until all requests are completed. From the perspective of request groups, the overall scheduling process consists of three stages:

1) Length Filtering: Speculative requests are maintained in a high-priority queue Q_{spec} and scheduled using an SFS policy, which always selects the request with the smallest amount of generated tokens. This ensures that short requests complete early and exit the queue quickly, while long requests remain and reveal themselves as potential long-tail candidates.

2) Length Estimation Update: The Context Manager maintains an estimated output length $\widehat{L}_g$ for each prompt group g.

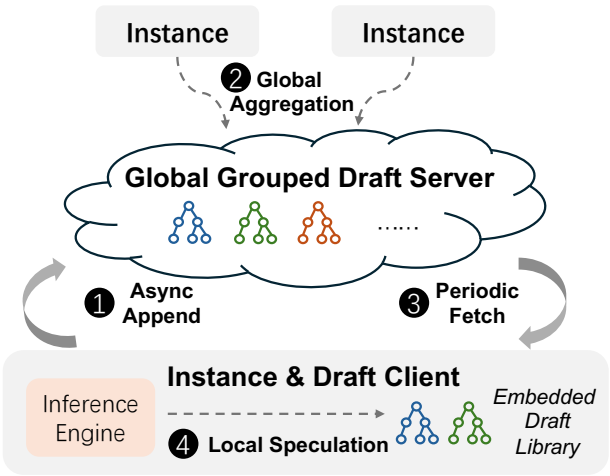


Figure 6: The distributed grouped draft server.

This value is dynamically updated as the maximum generation length among all completed requests in the group. If none of the requests in a group have completed yet, the group is classified as a potential long-tail candidate, and its estimated output length is conservatively set to the original `max_tokens` limit.

3) Scheduling Requests: Seer first prioritizes the execution of speculative requests to obtain length estimations. Once all speculative requests are in-flight or completed, Seer switches to an approximate LFS scheduling policy that prioritizes groups with longer predicted generation lengths $\widehat{L}_g$.

Thanks to the global KVCache pool, requests can be temporarily stored in the Request Buffer awaiting scheduling without loss between chunk executions, enabling more flexible scheduling based on continuously updated length context. To mitigate the potential negative impact of length prediction skewness, the scheduler randomly schedules requests from groups that have received the least execution time, and updates length estimates conservatively using the maximum generation length observed so far. Experiments in §6.4.1 demonstrate that context-aware scheduling achieves performance close to an oracle LFS scheduler that knows all output lengths in advance. Furthermore, the adaptive grouped speculative decoding technique, described in §4.4, provides additional acceleration for outlier long-tail requests.

4.4 Adaptive Grouped Speculative Decoding

To further improve resource utilization during the rollout phase, especially in the long-tail stage, Seer implements **Adaptive Grouped Speculative Decoding**, which **adaptively** adjusts draft lengths based on computational intensity and exploits the **grouped** pattern context (as analyzed in §3.3) to improve acceptance rates.

Seer introduces the Distributed Grouped Draft Server (DGDS), a distributed framework that shares speculative context across requests and instances. The core data structure of DGDS is the Compressed Suffix Tree (CST), which efficiently aggregates contextual statistics from multiple sequences and provides draft tokens with low complexity[1]. DGDS is designed to aggregate sequence context across requests within the same group into a unified CST, thereby providing high-quality speculative tokens to all inference instances.

To minimize speculative decoding latency in the critical path, DGDS adopts a distributed master-worker architecture, as illustrated in Figure 6. The system operates through four key steps:

1) Asynchronous Append: Each inference instance runs an independent process to handle output tokens, sending newly generated tokens to DGDS identified by `group_id`. To reduce communication overhead, each request batches a certain number of tokens before sending updates, which has a negligible impact on draft token quality.

2) Global Aggregation: DGDS aggregates token updates from requests belonging to the same group. To prevent cross-request interference, DGDS isolates updates by `request_id`, mapping each new token only to the corresponding local path in the CST.

3) Periodic Fetch: Each inference instance embeds a draft client as a library component. This client periodically synchronizes the latest CST from DGDS. To reduce communication overhead, the client only fetches CSTs corresponding to requests currently being processed on its instance and supports incremental synchronization.

4) Local Speculation: Inference instances perform speculation based on their local CSTs, which aggregate paths from all requests in the same group, allowing instances to share contextual statistics and obtain higher-quality draft tokens.

During the speculation phase, DGDS introduces an adaptive speculation range mechanism to dynamically balance inference latency and prediction accuracy. For different model architectures, such as dense and Mixture-of-Experts (MoE) models, we pre-compute distinct speculation token thresholds for individual requests and for the overall batch based on their computational characteristics. According to these thresholds, instances dynamically calculate the maximum draft length based on current concurrency levels. When generating draft tokens, the system also filters candidates according to token occurrence frequency and confidence scores provided by the CST to improve the acceptance rate. This adaptive control maintains decoding accuracy while maximizing the overall throughput and efficiency of speculative decoding across distributed rollout instances.

For long-tail requests, adaptive grouped speculative decoding offers two particular advantages. First, during the long-tail stage, concurrency is minimal, allowing larger draft lengths

[1]The complexity is $O(p+s)$, where p denotes the matching pattern length and s denotes the number of speculative tokens.

to increase the number of accepted tokens per request. We also implement multi-path speculative decoding to further improve the acceptance length in the long-tail stage. Second, as more requests in the same group complete over time, the CST aggregates richer contextual information. As demonstrated in Table 2, this enables long-tail requests to achieve substantially longer acceptance lengths.

5 Implementation

RL Infrastructure. Seer is a synchronous, colocated RL system designed to substantially reduce rollout long-tail latency and improve overall system throughput without compromising algorithmic fidelity. The training phase leverages Megatron [33] for distributed model training, while the rollout phase employs vLLM [16] for efficient inference. To minimize the overhead of transitioning between training and rollout phases, Seer implements an in-memory checkpoint conversion mechanism and a dedicated checkpoint engine [1] that enables efficient weight distribution and updates across the cluster.

Seer implements asynchronous reward computation to further improve system efficiency. Upon completion of each rollout request, the generated trajectories are immediately dispatched to a dedicated reward system for reward calculation. The rollout and reward resources can elastically scale based on workload demand. The reward system can either share resources with the rollout infrastructure or be deployed on a separate cluster and shared across multiple RL experiments to maximize resource utilization. This modular design ensures that Seer can dynamically adapt to varying resource availability while maintaining high throughput.

Global KVCache Pool. Seer leverages the Global KVCache Pool to enable efficient divided rollout. The KVCache pool employs a two-tier DRAM/SSD architecture that provides KVCache storage for all divided requests during each rollout iteration. To reduce transfer overhead, the KVCache server proactively prefetches KVCache to the target instance based on the request queue information in the Request Buffer. Once a rollout request completes, its corresponding KVCache is immediately released to maximize available DRAM storage capacity.

Speculative Decoding. Seer introduces DGDS to enable speculative decoding with context sharing. DGDS consists of two modules: the grouped draft server and the draft client. The grouped draft server operates as an independent RPC server that aggregates CST across multiple rollout instances. Its primary APIs are summarized in Table 3. To prevent load imbalances in communication and storage, Seer supports a multi-server deployment mode where clients automatically route requests to servers based on request group hashing. The primary APIs of the draft client are summarized in Table 4. Draft clients asynchronously fetch the CSTs for registered requests on their local instances via `fetch_cst` and execute speculation locally. To further reduce speculation overhead, the draft client is deployed as an embedded library within the same process as the inference engine and provides a zero-copy batch interface.

To achieve higher acceptance rates and longer acceptance lengths, Seer implements multi-path drafts, a variant of tree-based drafting [41]. An adaptive scheduling algorithm dynamically selects the optimal draft length and number of paths based on current system load and per-request acceptance rates. Within the suffix tree, Seer computes scores using suffix probabilities to filter low-probability candidates and returns multiple candidate paths via a beam-search mechanism. On the inference engine side, Seer employs cascade attention to efficiently process multi-path drafts for models with Grouped Query Attention [2] (GQA). For models with high arithmetic intensity attention mechanisms, such as Multi-head Latent Attention [22] (MLA), Seer simply processes them as multiple attention operations.

6 Evaluation

In this section, we evaluate Seer's performance advantages during the rollout phase, particularly in improving throughput and reducing tail latency (§6.2). In the ablation study (§6.3), we validate the advantages of divided rollout, context-aware scheduling, and adaptive grouped speculative decoding in accelerating long-generation rollout tasks. Finally, we present extended studies (§6.4) on scheduling and speculative decoding to demonstrate the effectiveness of our context-aware algorithms.

6.1 Setup

Testbed. Our experimental infrastructure consists of 32 high-performance compute nodes, each equipped with 8 H800 GPUs. For deployment, we adopt a strategy that balances task performance with resource efficiency by configuring different numbers of GPUs and parallelization strategies to serve models based on their size and architecture.

Models and Workloads. We use the GRPO [29] algorithm in our experiments. To validate the generalizability of Seer's design, we evaluate three models with diverse sizes and output characteristics: Moonlight [25], Qwen2-VL-72B [42], and Kimi-K2 [36]. Their configurations and RL workload characteristics are shown in Table 5. We employ Qwen2-VL-72B with tensor parallelism (TP8) and Kimi-K2 with data parallelism (DP32) and expert parallelism (EP32).

Baselines. We use veRL [32], a synchronous training system with colocated training and rollout, as the baseline. In terms of RL algorithm logic, Seer is identical to synchronous veRL, where each training iteration uses exclusively the data

Table 3: Grouped draft server API.

Operation	Description	Parameters
update_cst	Append generated tokens from a specific request to the compressed suffix tree	const string& group_id, int request_id, int prev_token_count, const vector<int>& new_tokens
fetch_cst	Fetch incremental draft contexts of request groups based on their current cache states (if have)	const vector<string>& group_ids, const vector<DraftCacheInfo>& draft_cache_infos

Table 4: Draft client API.

Operation	Description	Parameters
register_group	Register a new request group for draft fetching with TTL	const string& group_id, int ttl_seconds
batch_speculate	Generate speculative tokens for multiple requests via zero-copy memory access: reads input token patterns and writes predicted tokens directly to inference engine memory buffers	const vector<string>& group_ids, const vector<size_t>& buffer_offsets, void* pattern_buffer_ptr, void* output_buffer_ptr, const vector<SpeculationArgs>& speculation_args
SpeculationArgs: max_spec_tokens (int), pattern_lookup_max (int), pattern_lookup_min (int), top_k (int)		

Table 5: Model configurations and RL workload characteristics.

Metric	Moonlight	Qwen2-VL-72B	Kimi-K2
Model Size	32 GB	146 GB	1 TB
Total GPUs	32	128	256
GPUs per Instance	1	8	32
Prompts per Iter	400	600	800
Reqs per Prompt	8	16	8
Max. Gen. Length	65536	40960	98304

generated from the current rollout iteration, thus producing identical training results. To eliminate confounding factors, we employ an in-house implementation of vLLM [16] as the unified inference engine for both veRL and Seer.

Evaluation Metrics. Since Seer primarily addresses performance issues in synchronous rollout, our evaluation focuses exclusively on rollout-phase metrics, specifically the throughput (or completion time) of each rollout iteration.

6.2 End-to-End Performance

In our end-to-end experiments, we conduct three RL tasks based on the aforementioned models, with workload configurations detailed in Table 5. We focus on Seer's end-to-end performance during the rollout phase, specifically measuring rollout throughput. Notably, to validate Seer's effectiveness in mitigating tail latency, we analyze the tail latency phenomenon in our experiments and compare the tail latency between Seer and the baseline system in §6.2.2.

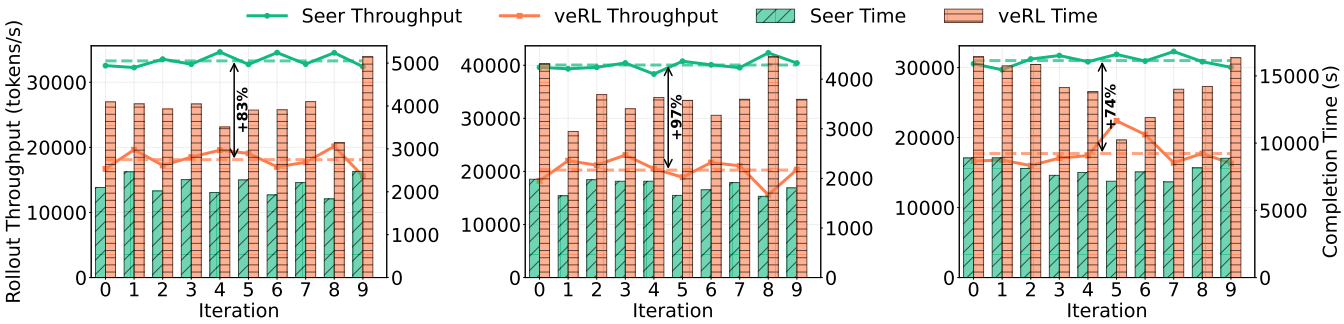
6.2.1 Rollout Throughput

We compare the throughput performance of Seer against the synchronous RL system, veRL. We run 10 iterations for each task and measure the rollout completion time and throughput for each iteration, with results presented in Figure 7. Despite significant variations in model size and workload characteristics across different RL tasks, Seer consistently achieves substantial speedups across all tasks, with throughput improvements ranging from 74% to 97% over veRL. This improvement stems from Seer's fine-grained load balancing and its ability to learn context information of co-grouped requests online, enabling superior scheduling strategies and more efficient grouped speculative decoding.

As shown in Figure 7, the baseline system exhibits greater variability in completion time and throughput across iterations. This instability arises from its group-level scheduling approach, where resource utilization is heavily influenced by the randomness of initial request assignments. In contrast, Seer employs fine-grained scheduling and dynamic load balancing, substantially reducing variability in resource utilization.

6.2.2 Long-Tail Time

To further analyze the source of Seer's performance advantages, we conduct a statistical analysis of the tail latency phenomenon during the rollout process. We define tail requests as the last 10% of requests to complete in a synchronous system rollout, and tail latency as the time spent **solely** processing these tail requests. Figure 8 shows the average tail latency and total execution time for the three tasks. The results demonstrate that tail latency is a severe problem in rollout,



(a) Moonlight (b) Qwen2-VL-72B (c) Kimi-K2

Figure 7: Rollout throughput (output tokens per second) and completion time comparison across three tasks. The dashed lines represent the average throughput over 10 iterations. Seer demonstrates significant throughput improvements over synchronous veRL, ranging from 74% to 97% across different workloads.

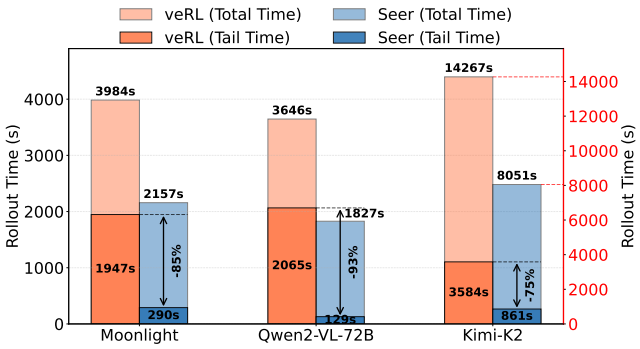


Figure 8: Tail latency and total time of three RL tasks.

Table 6: Performance improvement breakdown across three RL tasks. Context Sched. denotes context-aware scheduling (§4.3), and Grouped SD denotes adaptive grouped speculative decoding (§4.4). To minimize the impact of systematic variance across iterations, we evaluate the 5th iteration (out of 10) for each task.

Method	Moonlight	Qwen2-VL-72B	Kimi-K2
Baseline	1.00	1.00	1.00
+ Divided Rollout	1.27×	1.31×	1.35×
+ Context Sched.	1.33×	1.44×	1.47×
+ Grouped SD	1.77×	1.87×	1.77×

particularly for memory-constrained tasks like Moonlight and Qwen2-VL-72B, where the last 10% of requests alone consume up to 50% of the total execution time.

The tail latency problem stems from two primary causes. First, without length information, request queuing and preemption lead to delayed scheduling of long-output requests, causing a small number of requests to dominate the final execution phase. Second, monolithic request groups result in load imbalance across instances, where some instances are assigned requests with extremely long average lengths, leading to tail latency. By leveraging online context learning and fine-grained request scheduling, Seer significantly reduces tail latency by 75% to 93%, thereby substantially improving system throughput. A detailed breakdown of the contribution of each design component is presented in §6.3.

6.3 Improvement Breakdown

We present a systematic ablation study to quantify the contribution of each optimization component to Seer's overall performance. As an efficient synchronous RL system, Seer's design objectives center on maximizing rollout resource utilization and minimizing tail latency through a synergistic combination of techniques spanning memory management, load balancing, request scheduling, and inference acceleration. Table 6 demonstrates the cumulative end-to-end speedup obtained by incrementally integrating each major optimization component.

First, Seer's divided rollout mechanism enables dynamic, fine-grained load balancing across instances, mitigating tail latency caused by inter-instance load imbalance and reducing preemption overhead within instances due to varying KV-Cache memory consumption. This optimization yields significant improvements for memory-constrained tasks, achieving up to 35% throughput improvement.

Building upon divided rollout, context-aware scheduling leverages learned intra-group length distributions to guide request prioritization. By adopting an approximate LFS strategy, this approach mitigates tail latency induced by the delayed execution of long-running requests. This optimization provides up to 13% additional throughput improvement. Figure 9 illustrates the impact of context-aware scheduling on system behavior, demonstrating that Seer achieves consistently high KVCache memory utilization throughout the entire rollout

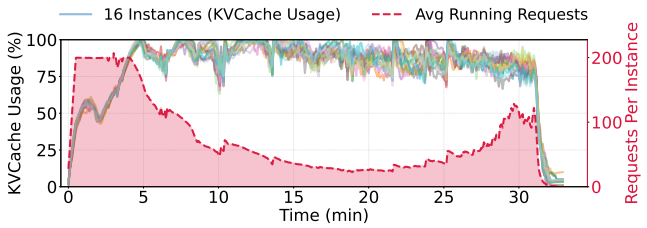
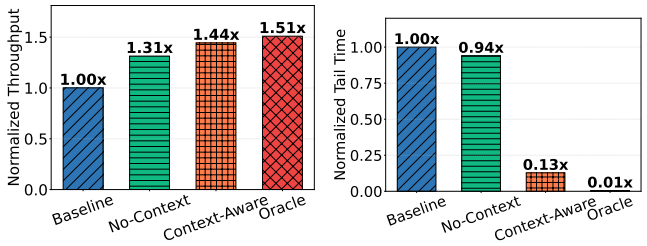


Figure 9: KVCache utilization and average running requests with Seer during a rollout iteration of the Qwen2-VL-72B task. Seer maintains consistently high memory utilization throughout the rollout process while substantially reducing the tail phase duration compared to the baseline (Figure 3a).



(a) Normalized rollout throughput.

(b) Normalized tail latency (defined in §6.2.2).

Figure 10: Impact of length context on improving throughput and reducing tail latency. No-Context applies only divided rollout without using length context to guide scheduling. Oracle obtains all output lengths in advance and applies the LFS strategy.

process while substantially reducing the tail phase duration compared to the baseline shown in Figure 3a.

To further address the computational underutilization inherent in tail requests, Seer incorporates adaptive grouped speculative decoding that dynamically adjusts both speculation depth and width based on runtime characteristics. This technique exploits the pattern similarity among co-located requests to accelerate LLM inference. The speculative decoding component contributes an additional 30–44% performance improvement over the scheduling optimizations alone.

6.4 Extended Studies

6.4.1 Context-Aware Scheduling

To validate the effectiveness of length context, we conduct an ablation study by varying the length information available to Seer's scheduler. Specifically, we compare: (1) No-Context, which applies only divided rollout without using length context to guide scheduling, and (2) Oracle, which obtains the actual output lengths of all requests in advance and replays the rollout iteration using these precise lengths to guide scheduling. We conduct the experiment on the 5th rollout iteration

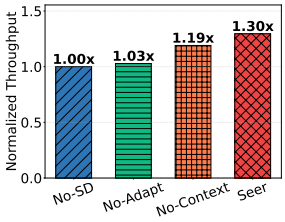


Figure 11: Normalized throughput on the 5th rollout iteration of the Qwen2-VL-72B task for different SD strategies. No-SD refers to disabling speculative decoding; No-Adapt refers to disabling adaptive scheduling; No-Context refers to disabling pattern context sharing within groups.

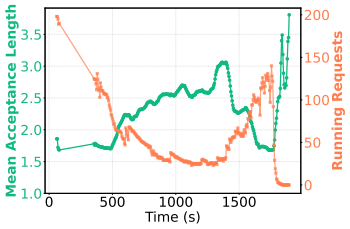


Figure 12: Evolution of smoothed mean acceptance length (averaged every 1 minute) throughout the rollout process for a selected instance of the Qwen2-VL-72B task.

of the Qwen2-VL-72B task, with throughput and tail latency comparisons shown in Figure 10.

While divided rollout significantly improves throughput through dynamic load balancing, the tail latency problem persists, with tail latency reduced by only 6% compared to the baseline. In contrast, context-aware scheduling leverages length prediction information from speculative requests to implement approximate longest-first scheduling, substantially reducing tail latency by 87% compared to the baseline. Compared to Oracle, context-aware scheduling achieves 95% of its throughput performance. This demonstrates that despite some performance degradation from prediction errors, context-aware scheduling still provides substantial benefits.

6.4.2 Adaptive Grouped Speculative Decoding

In this section, we address two key questions: (1) Is group context helpful for speculative decoding performance? (2) How does adaptive speculative decoding enhance system throughput?

Figure 11 presents the ablation study results for speculative decoding strategies. Compared to the baseline without SD, our adaptive grouped speculative decoding achieves a 30% improvement in end-to-end throughput. However, without group context sharing, this improvement drops to 19%, as the quality of draft tokens degrades, leading to shorter mean acceptance lengths.

When the adaptive strategy is disabled, speculative decoding only provides a marginal 3% improvement, nearly equivalent to not using it at all. This is because batch size and average request length vary dramatically during rollout. A fixed SD strategy easily leads to either severe compute bottlenecks or underutilization of computational resources. In

contrast, Seer employs a dynamic SD strategy that adaptively adjusts the number of draft tokens based on current system load and probabilities provided by the suffix draft tree. Figure 12 illustrates how the mean acceptance length evolves throughout the rollout process. Due to adaptive control, when short requests dominate and the batch size is large, the system generates fewer draft tokens, resulting in a lower mean acceptance length. Conversely, when long requests dominate and the batch size is small, more draft tokens are generated, leading to a higher mean acceptance length that reaches over 3.5 in the later stage. This adaptive behavior enables the system to preferentially accelerate long requests, thereby maximizing overall throughput.

7 Related Work

RL Frameworks for LLM Post-Training. Numerous open-source RL frameworks [15, 30, 32, 40, 43, 47, 54] have been proposed to achieve both efficiency and usability, providing robust infrastructure support for both research and production RL workloads. OpenRLHF [15] and veRL [32] seamlessly integrate training and inference engines, supporting various parallelism strategies to improve RL training efficiency. ROLL [43] targets large-scale RL training optimization, enabling flexible resource allocation and heterogeneous task scheduling. Slime [54] balances high performance with flexibility, supporting highly customizable data generation pipelines. These frameworks have been widely adopted, demonstrating strong applicability as RL infrastructure in both research and production environments. These works primarily focus on overall RL training workflow orchestration, while the long-tail problem in the rollout phase remains unaddressed.

On-Policy RL Optimization. Some efforts have been made to optimize the RL workflow at both the system and algorithmic levels. RealHF [26] dynamically reallocates model parameters and memory budgets during RLHF to optimize utilization. RLHFuse [52] proposes stage fusion to overlap reward and experience computation with the rollout long tail, thereby improving resource utilization. RLBoost [46] utilizes preemptible fragmented GPU resources to accelerate rollout at low cost. Nevertheless, these approaches do not adequately address the long-tail latency problem. For the long-tail latency problem in decoding, speculative decoding [17] (SD) is regarded as an effective optimization technique. A series of works [3, 18–20, 23] employ draft models or modules to generate draft tokens for target models, thereby reducing per-request latency. Despite achieving substantial speedup in small-batch scenarios, the effectiveness of these model-based speculative decoding algorithms is limited in RL scenarios, where model drift and the substantial overhead under large batch sizes result in significant performance degradation. Other works [7, 13, 24, 27] have proposed model-free SD methods. RhymeRL [13] and SPEC-RL [24] propose utilizing historical sequences generated from previous RL iterations as references to accelerate decoding. However, in most state-of-the-art model rollout scenarios, different iterations sample different data to improve generalization, leaving no historical sequences to reuse. In contrast, Seer exploits the similarity among requests within the same group, employing online context learning within a single iteration to enhance speculative decoding effectiveness.

Off-Policy RL Optimization. Recently, many works [6, 9, 11, 13, 31, 51, 53] have proposed RL workflows that do not maintain complete logical synchronization, trading a degree of off-policy behavior for improved RL efficiency. RhymeRL [13], APRIL [53], and RollPacker [9] reduce rollout tail latency through tail clipping and request repacking techniques. StreamRL [51], Areal [6], and Laminar [31] achieve complete decoupling of training and inference, substantially improving training concurrency and resource utilization. While asynchronous training improves RL efficiency, it introduces risks of instability and accuracy degradation, limiting its applicability in research and production environments where algorithmic fidelity is critical. Seer significantly reduces rollout tail latency through fine-grained scheduling of online context learning within the rollout phase, while maintaining consistency with on-policy algorithms.

8 Conclusion

In this paper, we present Seer, a synchronous RL system that accelerates the rollout process through online context learning. With divided rollout, a fine-grained and dynamically load-balanced scheduling approach, Seer leverages the similarity among requests within the same group in GRPO-like algorithms to achieve efficient scheduling and speculative decoding, while strictly maintaining consistency with on-policy RL algorithms. Experimental results demonstrate that Seer achieves 74-97% throughput improvement and 75-93% reduction in long-tail latency compared to the baseline system.

References

[1] Moontshot AI. Checkpoint engine. https://github.com/MoonshotAI/checkpoint-engine, 2025.

[2] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

[3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[4] Mark Chen. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Ke Cheng, Wen Hu, Zhi Wang, Peng Du, Jianguo Li, and Sheng Zhang. Enabling efficient batch serving for lmaas via generation length prediction. In *2024 IEEE International Conference on Web Services (ICWS)*, pages 853–864. IEEE, 2024.

[6] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.

[7] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.

[8] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. Efficient llm scheduling by learning to rank. *Advances in Neural Information Processing Systems*, 37:59006–59029, 2024.

[9] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, et al. Rollpacker: Mitigating long-tail rollouts for fast, synchronous rl post-training. *arXiv preprint arXiv:2509.21009*, 2025.

[10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.

[11] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, et al. Asyncflow: An asynchronous streaming rl framework for efficient llm post-training. *arXiv preprint arXiv:2507.01663*, 2025.

[12] Horace He and Thinking Machines Lab. Defeating nondeterminism in llm inference. *Thinking Machines Lab: Connectionism*, 2025. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

[13] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. History rhymes: Accelerating llm reinforcement learning with rhymerl. *arXiv preprint arXiv:2508.18588*, 2025.

[14] Jian Hu, Mingjie Liu, Ximing Lu, Fang Wu, Zaid Harchaoui, Shizhe Diao, Yejin Choi, Pavlo Molchanov, Jun Yang, Jan Kautz, et al. Brorl: Scaling reinforcement learning via broadened exploration. *arXiv preprint arXiv:2510.01180*, 2025.

[15] Jian Hu, Xibin Wu, Zilin Zhu, Weixun Wang, Dehao Zhang, Yu Cao, et al. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[17] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[18] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024.

[19] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

[20] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.

[21] Ziniu Li, Congliang Chen, Tianyun Yang, Tian Ding, Ruoyu Sun, Ge Zhang, Wenhao Huang, and Zhi-Quan Luo. Knapsack rl: Unlocking exploration of llms via optimizing budget allocation. *arXiv preprint arXiv:2509.25849*, 2025.

[22] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[23] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[24] Bingshuai Liu, Ante Wang, Zijun Min, Liang Yao, Haibo Zhang, Yang Liu, Anxiang Zeng, and Jinsong Su. Specrl: Accelerating on-policy reinforcement learning via speculative rollouts. *arXiv preprint arXiv:2509.23232*, 2025.

[25] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.

[26] Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. Real: Efficient rlhf training of large language models with parameter reallocation. *arXiv preprint arXiv:2406.14088*, 2024.

[27] Gabriele Oliaro, Zhihao Jia, Daniel F Campos, and Aurick Qiao. Suffixdecoding: Extreme speculative decoding for emerging ai applications. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[28] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, 2025.

[29] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[30] Gerald Shen, Zhilin Wang, Olivier Delalleau, Jiaqi Zeng, Yi Dong, Daniel Egert, Shengyang Sun, Jimmy Zhang, Sahil Jain, Ali Taghibakhshi, et al. Nemo-aligner: Scalable toolkit for efficient model alignment. *arXiv preprint arXiv:2405.01481*, 2024.

[31] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, et al. Laminar: A scalable asynchronous rl post-training framework. *arXiv preprint arXiv:2510.12633*, 2025.

[32] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

[33] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[34] Guijin Son, Hyunwoo Ko, Hoyoung Lee, Yewon Kim, and Seunghyeok Hong. Llm-as-a-judge & reward model: What they can and cannot do. *arXiv preprint arXiv:2409.11239*, 2024.

[35] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, pages 173–191, 2024.

[36] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.

[37] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1.5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.

[38] Meituan LongCat Team. Introducing longcat-flash-thinking: A technical report. *arXiv preprint arXiv:2509.18883*, 2025.

[39] vLLM Team. vllm: A high-throughput and memory-efficient inference and serving engine for llms. https://github.com/vllm-project/vllm, 2025.

[40] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. Trl: Transformer reinforcement learning. https://github.com/huggingface/trl, 2020.

[41] Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. Opt-tree: Speculative decoding with adaptive draft tree structure. *Transactions of the Association for Computational Linguistics*, 13:188–199, 2025.

[42] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.

[43] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, et al. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library. *arXiv preprint arXiv:2506.06122*, 2025.

[44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[45] Peter Weiner. Linear pattern matching algorithms. In *14th Annual Symposium on Switching and Automata Theory (swat 1973)*, pages 1–11. IEEE, 1973.

[46] Yongji Wu, Xueshen Liu, Haizhong Zheng, Juncheng Gu, Beidi Chen, Z Morley Mao, Arvind Krishnamurthy, and Ion Stoica. Rlboost: Harvesting preemptible resources for cost-efficient reinforcement learning on llms. *arXiv preprint arXiv:2510.19225*, 2025.

[47] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, et al. Deepspeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales. *arXiv preprint arXiv:2308.01320*, 2023.

[48] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[49] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

[50] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

[51] Yinmin Zhong, Zili Zhang, Xiaoniu Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, et al. Streamrl: Scalable, heterogeneous, and elastic rl for llms with disaggregated stream generation. *arXiv preprint arXiv:2504.15930*, 2025.

[52] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, et al. Optimizing {RLHF} training for large language models with stage fusion. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 489–503, 2025.

[53] Yuzhen Zhou, Jiajun Li, Yusheng Su, Gowtham Ramesh, Zilin Zhu, Xiang Long, Chenyang Zhao, Jin Pan, Xiaodong Yu, Ze Wang, et al. April: Active partial rollouts in reinforcement learning to tame long-tail generation. *arXiv preprint arXiv:2509.18521*, 2025.

[54] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. https://github.com/THUDM/slime, 2025. GitHub repository. Corresponding author: Xin Lv.